

Assignment No. 2 – Unit 2: Introducing Servlet

Concept – Servlet Life Cycle:

web.xml (Deployment Descriptor):

LifeCycleServlet.java:

[illegible]

```
Hello from GenericServlet!  
This servlet extends GenericServlet.  
GenericServlet is protocol-independent.
```

Server Info: Apache Tomcat/10.0
Servlet Name: HelloGenericServlet

Note: GenericServlet vs HttpServlet: GenericServlet is for any protocol; HttpServlet extends GenericServlet specifically for HTTP protocol with doGet()/doPost() methods.

Q3. Java program using HttpServlet to handle HTTP GET and POST requests.

Concept:

HttpServlet provides **doGet()** for GET requests and **doPost()** for POST requests. GET parameters are visible in URL; POST sends data in HTTP body (more secure).

HTML Form (index.html):

```
<!DOCTYPE html>
<html>
<body>
  <h2>GET Request Form</h2>
  <form action="HttpServletDemo" method="GET">
    Name: <input type="text" name="username"> <br><br>
    <input type="submit" value="Send GET">
  </form>

  <h2>POST Request Form</h2>
  <form action="HttpServletDemo" method="POST">
    Name: <input type="text" name="username"> <br>
    Password: <input type="password" name="password"> <br><br>
    <input type="submit" value="Send POST">
  </form>
</body>
</html>
```

HttpServletDemo.java:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HttpServletDemo extends HttpServlet {

    // Handles HTTP GET requests
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String username = request.getParameter("username");

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2 style='color:blue'>HTTP GET Request Received</h2>");
        out.println("<p>Method      : " + request.getMethod() + "</p>");
        out.println("<p>Username    : " + username + "</p>");
        out.println("<p>Query String: " + request.getQueryString() + "</p>");
        out.println("<p><i>GET params are visible in URL.</i></p>");
        out.println("</body></html>");
    }

    // Handles HTTP POST requests
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```

String username = request.getParameter("username");
String password = request.getParameter("password");

out.println("<html><body style='font-family:Arial;'>");
out.println("<h2 style='color:green'>HTTP POST Request Received</h2>");
out.println("<p>Method : " + request.getMethod() + "</p>");
out.println("<p>Username: " + username + "</p>");
out.println("<p>Password: ***** (hidden for security)</p>");
out.println("<p><i>POST params are NOT visible in URL.</i></p>");
out.println("</body></html>");
}
}

```

GET vs POST Comparison:

Feature	GET	POST
Data in URL	Yes (visible)	No (hidden in body)
Security	Less secure	More secure
Data Limit	~2048 chars	No limit
Bookmarkable	Yes	No
Use Case	Fetch/Search	Login, Forms, Upload

Q4. Servlet program to demonstrate the use of the RequestDispatcher interface with forward() method.

Concept:

RequestDispatcher is used to forward or include another servlet/JSP from the current servlet. **forward()** transfers control completely — the original servlet stops processing and the target takes over. The URL in browser does NOT change (server-side redirect).

FirstServlet.java (Dispatcher):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Set an attribute to pass to next servlet
        request.setAttribute("username", "Rahul Sharma");
        request.setAttribute("course", "Advanced Java");

        System.out.println("FirstServlet: Forwarding request to SecondServlet...");

        // Get RequestDispatcher pointing to SecondServlet
        RequestDispatcher rd = request.getRequestDispatcher("/SecondServlet");

        // forward() - transfers control to SecondServlet
        // Response NOT committed yet - SecondServlet will write the response
        rd.forward(request, response);

        // Code below will NOT execute after forward()
        System.out.println("This line will NOT be printed.");
    }
}

```

SecondServlet.java (Target):

```

import java.io.*;
import javax.servlet.*;

```

```

import javax.servlet.http.*;

public class SecondServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Retrieve attributes set by FirstServlet
        String username = (String) request.getAttribute("username");
        String course = (String) request.getAttribute("course");

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2 style='color:darkgreen'>SecondServlet (after forward)</h2>");
        out.println("<p>This page was forwarded from <b>FirstServlet</b>.</p>");
        out.println("<p>Username : <b>" + username + "</b></p>");
        out.println("<p>Course : <b>" + course + "</b></p>");
        out.println("<p>Note: URL in browser still shows /FirstServlet</p>");
        out.println("</body></html>");
    }
}

```

Note: After forward(), the browser URL remains /FirstServlet even though SecondServlet generated the response. This is different from response.sendRedirect().

Q5. Servlet program to demonstrate the use of RequestDispatcher with include() method.

Concept:

include() includes the response of another servlet/JSP into the current response. Unlike forward(), the original servlet retains control and can add content before AND after the included content.

MainServlet.java:

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class MainServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2 style='color:navy'>MainServlet Output</h2>");
        out.println("<p>--- Content from MainServlet (BEFORE include) ---</p>");
        out.println("<hr>");

        // Include content from HeaderServlet
        RequestDispatcher rd1 = request.getRequestDispatcher("/HeaderServlet");
        rd1.include(request, response);

        out.println("<hr>");
        out.println("<p>--- Back in MainServlet (AFTER include) ---</p>");

        // Include content from FooterServlet
        RequestDispatcher rd2 = request.getRequestDispatcher("/FooterServlet");
        rd2.include(request, response);

        out.println("</body></html>");
    }
}

```

```
}
```

HeaderServlet.java (Included):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HeaderServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        PrintWriter out = res.getWriter();
        out.println("<div style='background:#1a237e;color:white;padding:10px;'>");
        out.println("  <h3>Common Header (from HeaderServlet)</h3>");
        out.println("  <p>Welcome to Advanced Java Portal</p>");
        out.println("</div>");
    }
}
```

forward() vs include() Comparison:

Feature	forward()	include()
Control transfer	Complete - original servlet stops	Partial - original servlet continues
Response writing	Only target servlet writes	Both original and target write
Use case	Login, authentication	Headers, footers, navigation
URL change	No change in browser	No change in browser
After call	Cannot write more output	Can write more output

Q6. Servlet program to implement session tracking using Cookies.

Concept:

A **Cookie** is a small piece of data stored on the client's browser. The server sends a cookie with Set-Cookie header; the browser returns it with every request. Cookies can persist even after browser is closed (persistent cookies).

SetCookieServlet.java (Login – sets cookie):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SetCookieServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");

        // Create a Cookie with name and value
        Cookie userCookie = new Cookie("username", username);

        // Set cookie expiry time (in seconds): 60*60*24 = 1 day
        userCookie.setMaxAge(60 * 60 * 24);

        // Add cookie to the response (sent to browser)
        response.addCookie(userCookie);

        // Create another cookie
        Cookie roleCookie = new Cookie("role", "student");
        roleCookie.setMaxAge(60 * 60);
        response.addCookie(roleCookie);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h3>Cookies Set Successfully!</h3>");
        out.println("<p>Username cookie: " + username + "</p>");
        out.println("<a href='GetCookieServlet'>Go to Profile Page</a>");
        out.println("</body></html>");
    }
}
```

GetCookieServlet.java (Profile – reads cookie):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class GetCookieServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2>Profile Page (Reading Cookies)</h2>");

        // Get all cookies from request
        Cookie[] cookies = request.getCookies();
```

```

        if (cookies != null) {
            for (Cookie c : cookies) {
                out.println("<p>Cookie Name : <b>" + c.getName() + "</b></p>");
                out.println("<p>Cookie Value : <b>" + c.getValue() + "</b></p>");
                out.println("<hr>");
            }
        } else {
            out.println("<p>No cookies found!</p>");
        }

        // Delete a cookie (set maxAge to 0)
        out.println("<a href='DeleteCookieServlet'>Logout (Delete Cookie)</a>");
        out.println("</body></html>");
    }
}

```

Advantages and Disadvantages of Cookies:

Advantages:

- + Simple to use; no server-side memory needed
- + Can persist across browser sessions (persistent cookies)
- + Supported by all browsers

Disadvantages:

- Stored on client - user can disable or delete cookies
- Limited size (4KB per cookie)
- Sent with EVERY request - increases bandwidth
- Security risk: can be stolen via XSS attacks
- Not suitable for storing sensitive data

Q7. Servlet program to implement session tracking using Hidden Form Fields.

Concept:

Hidden Form Fields are HTML input fields with type='hidden'. They are not visible to the user but are submitted with the form. Used to pass data between pages without cookies or session objects.

Page1Servlet.java (Login form):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Page1Servlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String course = request.getParameter("course");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h3>Step 2: Confirm Enrollment</h3>");
        out.println("<p>Hello <b>" + username + "</b>, confirm your course:</p>");

        // Pass data to next page using HIDDEN FORM FIELDS
        out.println("<form action='Page2Servlet' method='POST'>");
        // These fields are NOT visible to user but submitted with form
        out.println("  <input type='hidden' name='username' value='" + username + "'>");
        out.println("  <input type='hidden' name='course' value='" + course + "'>");
        out.println("  <p>Course: <b>" + course + "</b></p>");
        out.println("  <input type='submit' value='Confirm Enrollment'>");
        out.println("</form>");
        out.println("</body></html>");
    }
}

```



```
    }  
}
```

Page2Servlet.java (Receives hidden fields):

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
  
public class Page2Servlet extends HttpServlet {  
    @Override  
    protected void doPost(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
  
        // Retrieve values passed via hidden form fields  
        String username = request.getParameter("username");  
        String course = request.getParameter("course");  
  
        response.setContentType("text/html");  
        PrintWriter out = response.getWriter();  
  
        out.println("<html><body style='font-family:Arial;'>");  
        out.println("<h2 style='color:green'>Enrollment Confirmed!</h2>");  
        out.println("<p>Student : <b>" + username + "</b></p>");  
        out.println("<p>Course : <b>" + course + "</b></p>");  
        out.println("<p>(Data passed via hidden form fields)</p>");  
        out.println("</body></html>");  
    }  
}
```

Advantages and Disadvantages of Hidden Form Fields:

Advantages:

- + Works even when cookies are disabled
- + Simple to implement
- + No server-side memory needed

Disadvantages:

- Data visible in page source (View Source) - not secure
- Only works with form submission (POST/GET)
- Cannot persist between non-form-based navigations
- URL gets long if many hidden fields used

Q8. Servlet program to demonstrate URL Rewriting for session management.

Concept:

URL Rewriting appends session data (like user ID) to every URL as a query string parameter. When user clicks the link, the data is sent back to server. Works even when cookies are disabled.

LoginServlet.java:

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
  
public class LoginServlet extends HttpServlet {  
    @Override  
    protected void doPost(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
  
        String username = request.getParameter("username");  
        String userid = "U" + (int)(Math.random() * 1000); // generate user ID  
  
        response.setContentType("text/html");  
        PrintWriter out = response.getWriter();  
  
        // Append user data to URLs (URL Rewriting)
```

```

String profileURL = "ProfileServlet?username=" + username + "&userid=" + userid;
String settingsURL = "SettingsServlet?username=" + username + "&userid=" + userid;
String logoutURL = "LogoutServlet?username=" + username + "&userid=" + userid;

out.println("<html><body style='font-family:Arial;'>");
out.println("<h2>Welcome, " + username + "! (User ID: " + userid + ")</h2>");
out.println("<p>Session managed via <b>URL Rewriting</b>.</p>");
out.println("<p>Notice the username and userid in each URL below:</p>");
out.println("<ul>");
out.println("  <li><a href='" + profileURL + "'>My Profile</a></li>");
out.println("  <li><a href='" + settingsURL + "'>Settings</a></li>");
out.println("  <li><a href='" + logoutURL + "'>Logout</a></li>");
out.println("</ul>");
out.println("</body></html>");
}
}

```

ProfileServlet.java (reads URL params):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ProfileServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Read data from URL query string
        String username = request.getParameter("username");
        String userid = request.getParameter("userid");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2>Profile Page</h2>");
        out.println("<p>Username (from URL): <b>" + username + "</b></p>");
        out.println("<p>User ID (from URL): <b>" + userid + "</b></p>");
        out.println("<p>URL: ProfileServlet?username=" + username +
            "&userid=" + userid + "</p>");
        out.println("</body></html>");
    }
}

```

Note: URL Rewriting Disadvantage: Session data is visible in URL (security risk), URL becomes long, data lost if user copies URL without parameters, bookmarks may not work properly.

Q9. Servlet program using HttpSession to store and retrieve user information.

Concept:

HttpSession stores user data on the server side. The server creates a session object and sends a session ID (JSESSIONID) to browser via cookie. Each request sends this ID back, allowing server to identify the user.

LoginServlet.java (stores in session):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
    }
}

```

```

String email    = request.getParameter("email");
String role     = request.getParameter("role");

// Create or get existing HttpSession
HttpSession session = request.getSession(true);
// true = create new session if none exists

// Store user data in session as attributes
session.setAttribute("username", username);
session.setAttribute("email", email);
session.setAttribute("role", role);
session.setAttribute("loginTime", new java.util.Date().toString());

// Set session timeout (in seconds): 30 minutes
session.setMaxInactiveInterval(30 * 60);

System.out.println("Session created. ID: " + session.getId());

response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("<html><body>");
out.println("<h3>Login Successful!</h3>");
out.println("<p>Session ID: " + session.getId() + "</p>");
out.println("<a href='ProfileServlet'>View Profile</a>");
out.println("</body></html>");
}
}

```

ProfileServlet.java (retrieves from session):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ProfileServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get existing session (false = don't create new one)
        HttpSession session = request.getSession(false);

        if (session == null) {
            out.println("<h3>Session expired. Please login again.</h3>");
            out.println("<a href='login.html'>Login</a>");
            return;
        }

        // Retrieve attributes from session
        String username = (String) session.getAttribute("username");
        String email    = (String) session.getAttribute("email");
        String role     = (String) session.getAttribute("role");
        String loginTime = (String) session.getAttribute("loginTime");

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2 style='color:navy'>User Profile</h2>");
        out.println("<p>Username    : <b>" + username + "</b></p>");
        out.println("<p>Email      : <b>" + email    + "</b></p>");
        out.println("<p>Role       : <b>" + role     + "</b></p>");
        out.println("<p>Login Time : <b>" + loginTime + "</b></p>");
        out.println("<p>Session ID : " + session.getId() + "</p>");
        out.println("<p>Is New    : " + session.isNew() + "</p>");
        out.println("<a href='LogoutServlet'>Logout</a>");
    }
}

```

```
        out.println("</body></html>");
    }
}
```

LogoutServlet.java (invalidates session):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LogoutServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null) {
            String username = (String) session.getAttribute("username");
            session.invalidate(); // Destroys the session completely
            System.out.println("Session invalidated for: " + username);
        }
        response.sendRedirect("login.html");
    }
}
```

Q10. Java Servlet program to demonstrate session management and explain the advantages of HttpSession.

HttpSession Key Methods:

Method	Description
request.getSession(true)	Creates new session if not exists
request.getSession(false)	Returns null if session doesn't exist
session.setAttribute(key, val)	Store data in session
session.getAttribute(key)	Retrieve data from session
session.removeAttribute(key)	Remove specific attribute
session.invalidate()	Destroy the session completely
session.getId()	Get unique session ID (JSESSIONID)
session.isNew()	Check if session is newly created
session.setMaxInactiveInterval(sec)	Set session timeout in seconds
session.getCreationTime()	Get session creation time

Program demonstrating session info:

```
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SessionInfoServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");

        HttpSession session = request.getSession(true);
        session.setAttribute("name", name);
        session.setAttribute("visits",
            session.getAttribute("visits") == null ? 1 :
            (Integer)session.getAttribute("visits") + 1);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2>HttpSession Information</h2>");
        out.println("<table border='1' cellpadding='5'>");
        out.println("<tr><td>Name</td><td>" + session.getAttribute("name") + "</td></tr>");
        out.println("<tr><td>Session ID</td><td>" + session.getId() + "</td></tr>");
        out.println("<tr><td>Is New?</td><td>" + session.isNew() + "</td></tr>");
        out.println("<tr><td>Visit Count</td><td>" + session.getAttribute("visits") + "</td></tr>");
        out.println("<tr><td>Created</td><td>" + new Date(session.getCreationTime()) + "</td></tr>");
        out.println("<tr><td>Last Access</td><td>" + new Date(session.getLastAccessedTime()) + "</td></tr>");
        out.println("<tr><td>Timeout(sec)</td><td>" + session.getMaxInactiveInterval() + "</td></tr>");
        out.println("</table>");
        out.println("<a href='LogoutServlet'>Invalidate Session</a>");
        out.println("</body></html>");
    }
}
```

Advantages of HttpSession:

1. Server-side storage: Data stored on server - more secure than cookies
2. No size limit: Can store any Java object (unlike cookies - 4KB limit)
3. Works always: Does not depend on browser cookie settings

4. Automatic ID management: Server handles JSESSIONID cookie automatically
 5. Timeout management: Session auto-expires after inactivity period
 6. Type-safe: Stores Java objects directly (not just strings like cookies)
-

Q11. Servlet program to implement session management using URL Rewriting.

Note: This is an extended version of Q8. Below shows using `response.encodeURL()` - the recommended way to do URL rewriting in Java Servlets.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class URLRewritingServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");

        // Use HttpSession with URL Rewriting fallback
        HttpSession session = request.getSession(true);
        session.setAttribute("username", username);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // response.encodeURL() automatically appends ;jsessionid=...
        // if cookies are disabled (URL Rewriting fallback)
        String homeURL    = response.encodeURL("HomeServlet");
        String profileURL = response.encodeURL("ProfileServlet");
        String logoutURL  = response.encodeURL("LogoutServlet");

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2>Welcome, " + username + "</h2>");
        out.println("<p>Session ID: " + session.getId() + "</p>");
        out.println("<p>Links below use URL Rewriting (encodeURL):</p>");
        out.println("<ul>");
        out.println("    <li><a href='" + homeURL    + "'>Home</a></li>");
        out.println("    <li><a href='" + profileURL + "'>Profile</a></li>");
        out.println("    <li><a href='" + logoutURL  + "'>Logout</a></li>");
        out.println("</ul>");
        out.println("<p><small>URL rewriting appends ;jsessionid=XYZ when cookies disabled</small></p>");
        out.println("</body></html>");
    }
}
```

Note: `response.encodeURL()` is best practice — it uses cookies if available, falls back to URL rewriting if cookies are disabled.

Q12. Servlet program to store and retrieve user information using HttpSession.

Note: Extended version with complete HTML login form and full session lifecycle.

login.html (Frontend Form):

```
<!DOCTYPE html>
<html>
<head><title>Student Login</title></head>
<body style="font-family:Arial; max-width:400px; margin:50px auto;">
    <h2>Student Login Portal</h2>
    <form action="StudentLoginServlet" method="POST">
        <label>Name      </label><input type="text" name="name" required><br><br>
        <label>Roll No   </label><input type="text" name="rollno" required><br><br>
        <label>Course     </label>
        <select name="course">
            <option>BCA</option><option>MCA</option><option>BSc CS</option>
```

```

        </select><br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>

```

StudentLoginServlet.java:

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentLoginServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");
        String rollno = request.getParameter("rollno");
        String course = request.getParameter("course");

        // Create session and store all user info
        HttpSession session = request.getSession(true);
        session.setAttribute("studentName", name);
        session.setAttribute("studentRollNo", rollno);
        session.setAttribute("studentCourse", course);
        session.setMaxInactiveInterval(1800); // 30 min

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2 style='color:green'>Login Successful!</h2>");
        out.println("<p>Welcome, <b>" + name + "</b>!</p>");
        out.println("<a href='StudentDashboard'>Go to Dashboard</a>");
        out.println("</body></html>");
    }
}

```

StudentDashboard.java (retrieves session info):

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentDashboard extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        if (session == null || session.getAttribute("studentName") == null) {
            out.println("<h3>Please login first!</h3>");
            out.println("<a href='login.html'>Login</a>");
            return;
        }

        String name = (String) session.getAttribute("studentName");
        String rollno = (String) session.getAttribute("studentRollNo");
        String course = (String) session.getAttribute("studentCourse");

        out.println("<html><body style='font-family:Arial; max-width:500px; margin:auto;'>");
        out.println("<h2 style='color:navy'>Student Dashboard</h2>");
        out.println("<table border='1' cellpadding='8' width='100%'>");
        out.println("<tr style='background:#1a237e;color:white;'>" +

```

```
        "<th>Field</th><th>Value</th></tr>");
out.println("<tr><td>Name</td><td>" + name + "</td></tr>");
out.println("<tr><td>Roll No</td><td>" + rollno + "</td></tr>");
out.println("<tr><td>Course</td><td>" + course + "</td></tr>");
out.println("<tr><td>Session ID</td><td>" + session.getId() + "</td></tr>");
out.println("</table>");
out.println("<br><a href='LogoutServlet'>Logout</a>");
out.println("</body></html>");
    }
}
```

Q13. Servlet program that connects to a database using JDBC and displays all records from a table in the browser.

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DisplayStudentsServlet extends HttpServlet {

    static final String DB_URL = "jdbc:mysql://localhost:3306/college_db";
    static final String DB_USER = "root";
    static final String DB_PASS = "password";

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<!DOCTYPE html><html>");
        out.println("<head><title>All Students</title>");
        out.println("<style>");
        out.println("    body{font-family:Arial;margin:20px;}");
        out.println("    table{border-collapse:collapse;width:100%;}");
        out.println("    th{background:#1a237e;color:white;padding:10px;}");
        out.println("    td{padding:8px;border:1px solid #ddd;}");
        out.println("    tr:nth-child(even){background:#f2f2f2;}");
        out.println("</style></head><body>");
        out.println("<h2>Student Records</h2>");

        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
            stmt = conn.createStatement();
            rs = stmt.executeQuery("SELECT * FROM students ORDER BY id");

            out.println("<table>");
            out.println("<tr><th>ID</th><th>Name</th><th>Age</th><th>Course</th></tr>");

            int count = 0;
            while (rs.next()) {
                count++;
                out.println("<tr>");
                out.println("    <td>" + rs.getInt("id") + "</td>");
                out.println("    <td>" + rs.getString("name") + "</td>");
                out.println("    <td>" + rs.getInt("age") + "</td>");
                out.println("    <td>" + rs.getString("course") + "</td>");
                out.println("</tr>");
            }
            out.println("</table>");
            out.println("<p>Total Records: <b>" + count + "</b></p>");

        } catch (ClassNotFoundException e) {
            out.println("<p style='color:red'>Driver Error: " + e.getMessage() + "</p>");
        } catch (SQLException e) {
            out.println("<p style='color:red'>DB Error: " + e.getMessage() + "</p>");
        }
    }
}
```

```

    } finally {
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (conn != null) conn.close();
        } catch (SQLException e) { e.printStackTrace(); }
    }
    out.println("</body></html>");
}
}

```

Q14. Servlet program using HttpServlet and PreparedStatement to insert user input from a form into a database.

HTML Form (addStudent.html):

```

<!DOCTYPE html>
<html>
<body style="font-family:Arial; max-width:400px; margin:50px auto;">
    <h2>Add New Student</h2>
    <form action="AddStudentServlet" method="POST">
        Name : <input type="text" name="name" required><br><br>
        Age : <input type="number" name="age" required><br><br>
        Course: <input type="text" name="course" required><br><br>
        <input type="submit" value="Add Student">
        <input type="reset" value="Clear">
    </form>
</body>
</html>

```

AddStudentServlet.java:

```

import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class AddStudentServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Get form parameters
        String name = request.getParameter("name");
        int age = Integer.parseInt(request.getParameter("age"));
        String course = request.getParameter("course");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        Connection conn = null;
        PreparedStatement pstmt = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college_db", "root", "password");

            // Use PreparedStatement to prevent SQL injection
            String sql = "INSERT INTO students(name, age, course) VALUES(?, ?, ?)";
            pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, name);
            pstmt.setInt(2, age);
            pstmt.setString(3, course);

            int rows = pstmt.executeUpdate();

```

```

        out.println("<html><body style='font-family:Arial;'>");
        if (rows > 0) {
            out.println("<h2 style='color:green'>Student Added Successfully!</h2>");
            out.println("<p>Name : <b>" + name + "</b></p>");
            out.println("<p>Age : <b>" + age + "</b></p>");
            out.println("<p>Course: <b>" + course + "</b></p>");
        } else {
            out.println("<h2 style='color:red'>Failed to add student.</h2>");
        }

        } catch (NumberFormatException e) {
            out.println("<p style='color:red'>Invalid age: " + e.getMessage() + "</p>");
        } catch (ClassNotFoundException | SQLException e) {
            out.println("<p style='color:red'>Error: " + e.getMessage() + "</p>");
        } finally {
            try {
                if (pstmt != null) pstmt.close();
                if (conn != null) conn.close();
            } catch (SQLException e) { e.printStackTrace(); }
        }
        out.println("<a href='addStudent.html'>Add Another</a> | ");
        out.println("<a href='DisplayStudentsServlet'>View All</a>");
        out.println("</body></html>");
    }
}

```

Q15. Servlet program that retrieves data from a database and displays it using ResultSet in an HTML table.

Note: This extends Q13 with styled table, search filter, and ResultSet metadata.

```

import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentTableServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String filterCourse = request.getParameter("course"); // optional filter

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<!DOCTYPE html><html><head><title>Student Table</title>");
        out.println("<style>body{font-family:Arial;margin:20px}");
        out.println("<table{width:100%;border-collapse:collapse}");
        out.println("<th{background:#1565c0;color:#fff;padding:10px;text-align:left}");
        out.println("<td{padding:8px;border-bottom:1px solid #ddd}");
        out.println("<tr:hover{background:#e3f2fd}</style></head><body>");
        out.println("<h2>Student Database</h2>");
        out.println("<form method='GET'>");
        out.println("Filter by Course: <input name='course' value='" +
            (filterCourse != null ? filterCourse : "") + "'>");
        out.println("<input type='submit' value='Filter'>");
        out.println("<a href='StudentTableServlet'> [Show All]</a></form><br>");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college_db","root","password");
            String sql = (filterCourse != null && !filterCourse.isEmpty())

```

```

        ? "SELECT * FROM students WHERE course = ?"
        : "SELECT * FROM students ORDER BY id";

PreparedStatement pstmt = conn.prepareStatement(sql);
if (filterCourse != null && !filterCourse.isEmpty())
    pstmt.setString(1, filterCourse);

ResultSet rs = pstmt.executeQuery();

// Use ResultSetMetaData to get column names dynamically
ResultSetMetaData meta = rs.getMetaData();
int colCount = meta.getColumnCount();

out.println("<table><tr>");
for (int i = 1; i <= colCount; i++)
    out.println("<th>" + meta.getColumnName(i).toUpperCase() + "</th>");
out.println("</tr>");

int rowCount = 0;
while (rs.next()) {
    rowCount++;
    out.println("<tr>");
    for (int i = 1; i <= colCount; i++)
        out.println("<td>" + rs.getString(i) + "</td>");
    out.println("</tr>");
}
out.println("</table>");
out.println("<p><b>Total: " + rowCount + " record(s) found.</b></p>");

rs.close(); pstmt.close(); conn.close();

} catch (Exception e) {
    out.println("<p style='color:red'>Error: " + e.getMessage() + "</p>");
}
out.println("</body></html>");
}
}

```

Q16. Servlet program to update database records using JDBC based on user input from a form.

HTML Form (updateStudent.html):

```

<!DOCTYPE html>
<html>
<body style="font-family:Arial; max-width:400px; margin:50px auto;">
    <h2>Update Student Record</h2>
    <form action="UpdateStudentServlet" method="POST">
        Student ID (to update): <input type="number" name="id" required><br><br>
        New Name : <input type="text" name="name" required><br><br>
        New Age : <input type="number" name="age" required><br><br>
        New Course: <input type="text" name="course" required><br><br>
        <input type="submit" value="Update Student">
    </form>
</body>
</html>

```

UpdateStudentServlet.java:

```

import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class UpdateStudentServlet extends HttpServlet {
    @Override

```

```

protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    int    id    = Integer.parseInt(request.getParameter("id"));
    String name  = request.getParameter("name");
    int    age   = Integer.parseInt(request.getParameter("age"));
    String course = request.getParameter("course");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    Connection      conn = null;
    PreparedStatement pstmt = null;

    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password");

        // First check if record exists
        PreparedStatement check = conn.prepareStatement(
            "SELECT COUNT(*) FROM students WHERE id = ?");
        check.setInt(1, id);
        ResultSet rs = check.executeQuery();
        rs.next();
        int count = rs.getInt(1);
        check.close();

        if (count == 0) {
            out.println("<h3 style='color:red'>No student found with ID: " + id + "</h3>");
            return;
        }

        // Update the record
        String sql = "UPDATE students SET name=?, age=?, course=? WHERE id=?";
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, name);
        pstmt.setInt(2, age);
        pstmt.setString(3, course);
        pstmt.setInt(4, id);

        int rows = pstmt.executeUpdate();

        out.println("<html><body style='font-family:Arial;'>");
        out.println("<h2 style='color:green'>Record Updated Successfully!</h2>");
        out.println("<p>" + rows + " row(s) updated.</p>");
        out.println("<p>ID      : <b>" + id      + "</b></p>");
        out.println("<p>Name   : <b>" + name   + "</b></p>");
        out.println("<p>Age    : <b>" + age    + "</b></p>");
        out.println("<p>Course: <b>" + course + "</b></p>");

    } catch (Exception e) {
        out.println("<p style='color:red'>Error: " + e.getMessage() + "</p>");
    } finally {
        try {
            if (pstmt != null) pstmt.close();
            if (conn != null) conn.close();
        } catch (SQLException e) { e.printStackTrace(); }
    }

    out.println("<a href='updateStudent.html'>Update Another</a> | ");
    out.println("<a href='StudentTableServlet'>View All Records</a>");
    out.println("</body></html>");
}
}

```

Q17. Explain how Servlets interact with databases using JDBC, and discuss the advantages of Servlets over CGI for database-driven web applications.

Part A: How Servlets Interact with Databases using JDBC

Servlets interact with databases through the **JDBC (Java Database Connectivity)** API. The following are the standard steps involved:

```
STEP 1: Load the JDBC Driver
    Class.forName("com.mysql.cj.jdbc.Driver");

STEP 2: Establish Connection
    Connection conn = DriverManager.getConnection(url, user, password);

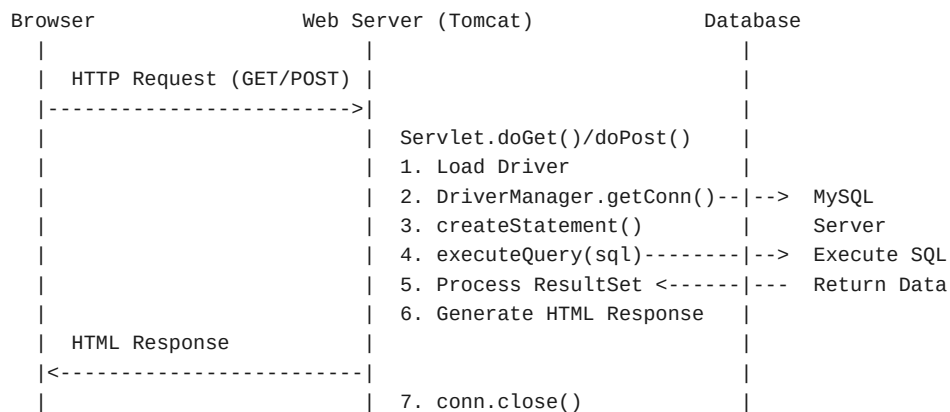
STEP 3: Create Statement / PreparedStatement
    Statement stmt      = conn.createStatement();
    PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM table WHERE id=?");

STEP 4: Execute SQL Queries
    ResultSet rs = stmt.executeQuery("SELECT * FROM students"); // SELECT
    int rows     = stmt.executeUpdate("INSERT INTO students..."); // INSERT/UPDATE/DELETE

STEP 5: Process Results (ResultSet)
    while (rs.next()) {
        String name = rs.getString("name");
        int    age  = rs.getInt("age");
    }

STEP 6: Close Resources
    rs.close(); stmt.close(); conn.close();
```

Complete Servlet-JDBC Architecture Diagram:



Part B: Advantages of Servlets over CGI

Feature	Servlet	CGI (Common Gateway Interface)
Process Model	Single JVM, multiple threads per request	New process created for EVERY request
Performance	High - threads are lightweight	Low - processes are heavy, slow startup
Memory	Shared memory between requests	Each process has separate memory
Persistent State	Can maintain state via HttpSession	Stateless - each request is independent
Platform	Platform independent (Java)	Platform dependent
Database Connection	Connection pooling possible (reuse)	New connection per process - expensive
Security	Java SecurityManager, HTTPS support	Less secure, OS-level permissions only
Portability	Runs on any Java-enabled server	Server and OS specific scripts
Object-Oriented	Full Java OOP, libraries available	Procedural (Perl, C, shell)

Why Servlets are Better for Database-Driven Applications:

1. CONNECTION POOLING:
Servlets can use JDBC Connection Pool (e.g., Apache DBCP, HikariCP).
Connection is reused across requests -> much faster than creating new connection per request (as CGI would do).
2. PERSISTENT OBJECTS:
Servlet stays in memory between requests. Database metadata, prepared statements, and config objects can be cached in `init()` and reused.
3. THREAD SAFETY WITH SHARED RESOURCES:
Multiple client requests handled by threads sharing the same servlet instance -> efficient resource use with proper synchronization.
4. SESSION MANAGEMENT:
`HttpSession` allows storing user-specific DB query results across multiple page requests - impossible with stateless CGI.
5. EXCEPTION HANDLING:
Java's try-catch-finally ensures proper resource cleanup.
CGI scripts often leave DB connections open on errors.

Example: CGI vs Servlet for 100 concurrent users

CGI: 100 processes created -> 100 DB connections -> server crash risk
Servlet: 1 servlet instance, 100 threads -> pool of 10 DB connections
shared -> efficient, fast, scalable

Features of Servlets (as per syllabus):

- | | |
|-------------------------|--|
| 1. Platform Independent | : Written in Java, runs on any OS with JVM |
| 2. Protocol Independent | : Can work with any protocol (HTTP, FTP, etc.) |
| 3. Powerful | : Full access to Java APIs (JDBC, JNDI, etc.) |
| 4. Persistent | : Loaded once, handles multiple requests |
| 5. Secure | : Java security model + HTTPS support |
| 6. Portable | : Deploy on Tomcat, JBoss, WebLogic, etc. |
| 7. Efficient | : Thread-based (not process-based like CGI) |
| 8. Extensible | : Can be extended with filters, listeners |

javax.servlet Package – Key Interfaces & Classes:

Interfaces:

Servlet	- Core servlet interface (init, service, destroy)
ServletRequest	- Represents client request
ServletResponse	- Represents server response
RequestDispatcher	- For forward() and include()
ServletConfig	- Servlet configuration
ServletContext	- Application-wide context

Classes:

GenericServlet	- Protocol-independent servlet
ServletInputStream	- For reading binary data from request
ServletOutputStream	- For writing binary data to response

javax.servlet.http Package – Key Classes:

HttpServlet	- HTTP-specific servlet (doGet, doPost)
HttpServletRequest	- HTTP request (getCookies, getSession, etc.)
HttpServletResponse	- HTTP response (sendRedirect, addCookie, etc.)
HttpSession	- Session management
Cookie	- Cookie creation and management